EduBridge AI

Full Architecture Plan

A stage-aware multilingual learning framework for progressive transition from regional languages to English

Item	Detail
Team	Bhavan Kalyan S, Vishak P
Guide	Dr. Sakthivel K
Department	CSBS, K. S. Rangasamy College of Technology, Namakkal
Document status	Draft v1.0 for project review
Scope covered	Phase 1 build (core MVP) in full, Phase 2 as defined extension points

Honesty statement. EduBridge AI is under development. No controlled field study has been run, so this document makes no claim of improved learning outcomes. BKT parameters and stage thresholds are untuned, configurable defaults.

Contents

1. Purpose, scope and design principles
2. System architecture
3. Technology stack
4. Repository structure
5. Language-stage model
6. Data architecture
7. Core services and the answer pipeline
8. API design
9. Dashboards and their features
10. Frontend architecture
11. Security, privacy and compliance
12. CI/CD and environments
13. Testing strategy
14. Phase 2 extension architecture
15. Delivery roadmap
16. Risks, limitations and open decisions

1. Purpose, scope and design principles

Students who learn in Tamil, Malayalam, Kannada or Telugu often struggle when academic content switches to English. Most platforms translate content but do not manage the transition. EduBridge AI moves each student from regional-language learning to English-only learning, one concept at a time, using mastery tracking to decide how much English support to give.

Scope

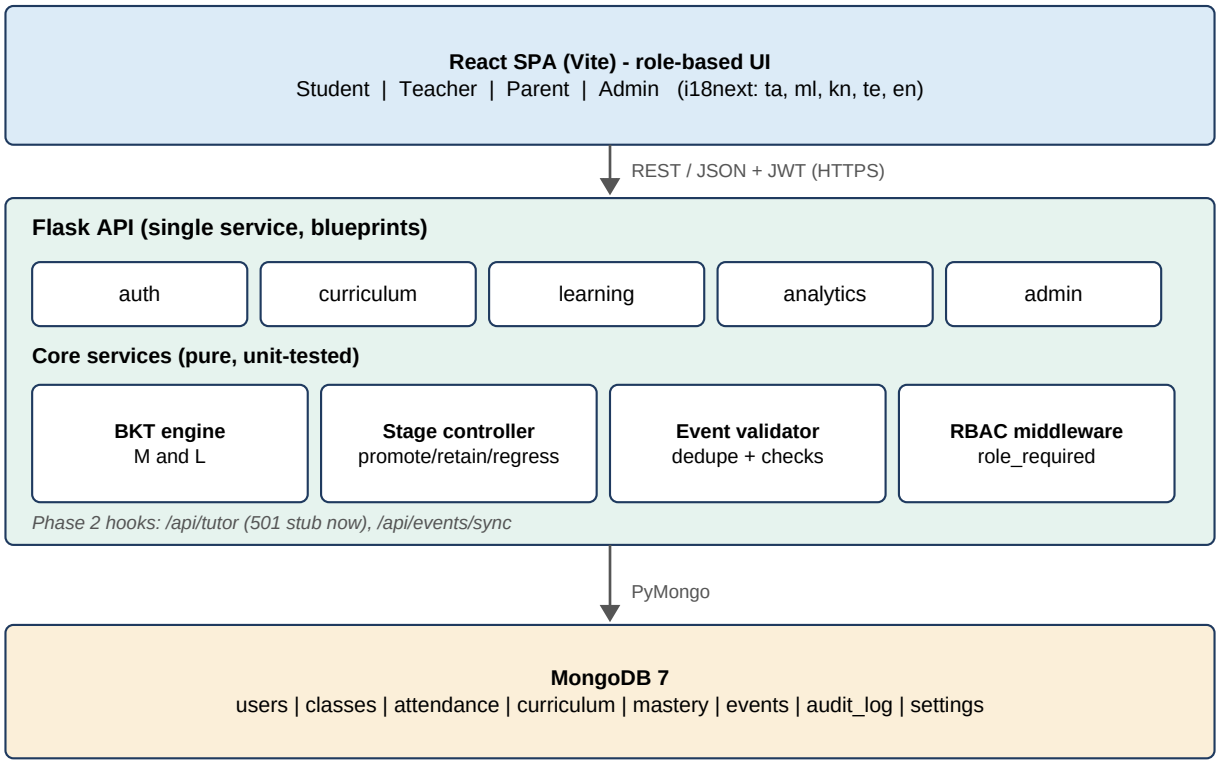
In scope (Phase 1)	Out of scope in Phase 1 (clean hooks only)
Authentication and four roles (student, teacher, parent, admin) Curriculum in 4 regional languages plus English Five-stage controller (rule based) Dual Bayesian Knowledge Tracing (concept M, keyword L) Event log with validation Lessons and quizzes Four role dashboards Docker, tests, CI	RAG tutor and any LLM call Embeddings and vector index Offline lesson packs and sync queue Support-score function P(u,c) and alerts Voice interaction Learned or tuned BKT parameters Multi-institution study

Design principles

- **Per-concept, not per-student.** A student can be at Stage 3 in one concept and Stage 0 in another.
- **Two kinds of mastery.** Knowing the concept (M) is tracked separately from knowing the English keywords (L).
- **Explainable rules over black boxes.** Teachers can see why a stage changed.
- **Configurable, never hard-coded.** BKT parameters and thresholds live in config and are documented as untuned.
- **Privacy by design.** Minimal data, consent flag, synthetic seed data, parents see progress only.
- **Honest claims.** The stage is the amount of English support, not English proficiency.
- **Simple first.** A single Flask service and one database suit a two-person team; split later only if needed.

2. System architecture

EduBridge AI is a three-tier web application. All server logic runs in one Flask service organised by blueprints, which keeps development and debugging simple and leaves room to add the AI and offline layers later.



Deployment: Docker containers (frontend, backend, mongo) via Docker Compose; images built by GitHub Actions.

Layer	Responsibility
Presentation (React SPA)	Role-based screens, language switching, quiz interaction, charts. Generates a client UUID for every answer.
Application (Flask API)	Authentication, permissions, curriculum delivery, answer processing, analytics, administration.
Domain services	BKT engine, stage controller, event validator. Pure functions with no database access, so they are easy to unit test.
Data (MongoDB)	Users, classes, attendance, curriculum, mastery, append-only events, audit log, settings.

3. Technology stack

Area	Choice	Reason
Frontend	React 18, Vite, React Router, Axios	Fast build, standard tooling
Internationalisation	i18next	UI strings in ta, ml, kn, te, en
Styling and charts	Tailwind CSS, Recharts	Quick responsive UI; heatmap and distribution charts
Fonts	Noto Sans Tamil, Malayalam, Kannada, Telugu	Correct rendering of all four scripts
Backend	Python 3.12, Flask, Flask-JWT-Extended	Matches the paper; simple blueprints
Validation	Pydantic	Typed request validation and consistent errors
Password hashing	bcrypt	Industry standard
Database	MongoDB 7 with PyMongo	Document model fits per-concept, per-language content
Testing	pytest, pytest-cov, Vitest, Playwright (small set)	Unit, API, component and end-to-end
Lint and audit	ruff, ESLint, pip-audit, npm audit, gitleaks	Quality and security gates in CI
Containers and CI	Docker, Docker Compose, GitHub Actions, GHCR	Reproducible builds and free-tier friendly

Free-tier hosting limits change often. Check current limits for the hosts you choose before committing to them.

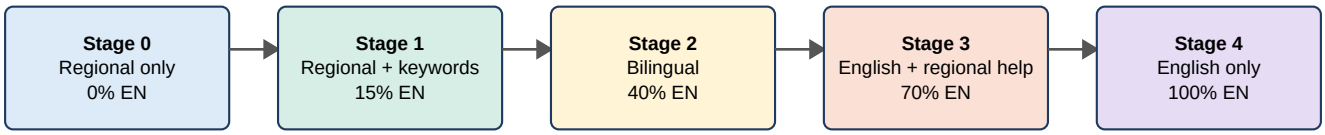
4. Repository structure

One monorepo is used so a single pull request can change the API and the UI together, with no cross-repository version drift.

```
edubridge-ai/
|-- backend/
|   |-- app.py, config.py, extensions.py
|   |-- blueprints/   auth.py curriculum.py learning.py analytics.py admin.py tutor.py(stub)
|   |-- services/     bkt.py stage_controller.py event_validator.py
|   |-- middleware/   rbac.py
|   |-- schemas/      Pydantic request/response models
|   |-- tests/        unit/  api/
|   |-- requirements.txt, Dockerfile, openapi.yaml
|-- frontend/
|   |-- src/api  src/auth  src/i18n  src/hooks  src/components
|   |-- src/pages/    student/ teacher/ parent/ admin/
|   |-- Dockerfile
|-- infra/            docker-compose.yml, docker-compose.staging.yml, nginx.conf
|-- scripts/          seed_curriculum.py, seed_users.py, healthcheck.sh
|-- .github/          workflows/(ci.yml, cd-staging.yml, cd-prod.yml), CODEOWNERS, PR template
|-- .env.example, README.md
```

5. Language-stage model

The paper defines five levels (Level 1 to 5). In code they are indexed 0 to 4, so Level 1 equals Stage 0.



Promotion moves right, regression moves left. Tracked per student PER CONCEPT.
Stage = amount of English support given, NOT a measure of English proficiency.

Stage	Paper level	What the student sees	English share
0	1	Full explanation in the regional language	0%
1	2	Regional explanation plus English keyword cards	15%
2	3	Bilingual explanation plus keywords	40%
3	4	English explanation with regional help	70%
4	5	English explanation only	100%

Each concept stores one authored content block per stage (s0 to s4) and per language. Phase 1 therefore needs no AI to produce stage-appropriate text; in Phase 2 the tutor can draft it for teacher review.

Content authoring plan: Tamil is written in full first. Malayalam, Kannada and Telugu are added as drafts marked TODO_REVIEW until a fluent teacher approves them.

6. Data architecture

Collections and their purpose

Collection	Key fields	Indexes and notes
users	name, role, email, password_hash, language, grade, class_id, linked_students[], consent, active	Unique email. Passwords bcrypt-hashed.
classes	class_id, name, teacher_id, student_ids[]	Teacher can access only own class.
attendance	user_id, date, present	Compound (user_id, date). Feeds the parent view.
curriculum	concept_id, subject, chapter, lesson, title{lang}, keywords[{en,ta,ml,kn,te}], content{lang:{s0..s4}}, quiz[{q_id, kind, question{lang}, options[], answer}], review_status	One document per concept. review_status: draft, reviewed, approved.
mastery	user_id, concept_id, M, L, stage, recent[] (last 5), last_correct_at, updated_at	Unique compound (user_id, concept_id).
events	event_id, user_id, concept_id, stage, kind, q_id, correct, time_ms, ts, valid	Append-only. Unique index on event_id gives duplicate protection and prepares for offline sync.
audit_log	actor_id, action, target, before, after, ts	Records admin and content changes.
settings	bkt{P0,T,g,s}, thresholds{promote_M, promote_L, regress_M, streak}	Editable by admin; labelled untuned.

Entity relationships

```
classes 1---* users(student)        users(teacher) 1---* classes
users(parent) *---* users(student)     (linked_students)
users(student) 1---* mastery *---1 curriculum(concept)
users(student) 1---* events  *---1 curriculum(concept)
users(student) 1---* attendance
```

Example documents

```
mastery: { "user_id": "u123", "concept_id": "math9_var_01", "M": 0.52, "L": 0.31,
           "stage": 1, "recent": [1,1,0,1,1], "last_correct_at": "2026-11-06T10:15:00Z" }

events:  { "event_id": "6f1c...-uuid", "user_id": "u123", "concept_id": "math9_var_01",
           "stage": 1, "kind": "keyword", "q_id": "q4", "correct": true, "time_ms": 8200,
           "ts": "2026-11-06T10:15:00Z", "valid": true }
```

7. Core services and the answer pipeline

7.1 BKT engine

Two independent Bayesian Knowledge Tracing instances run per student per concept: M for the concept and L for the English keywords. After each valid answer the probability of knowing is updated with guess (g) and slip (s), then the learning rate (T) is applied.

```
if answer correct:   P' = P*(1-s) / ( P*(1-s) + (1-P)*g )
if answer incorrect: P' = P*s     / ( P*s     + (1-P)*(1-g) )
P_next = P' + (1 - P') * T

Starting defaults (UNTUNED, in config.py, overridable by environment variables):
P0 = 0.30   T = 0.10   g = 0.20   s = 0.10
```

7.2 Stage controller

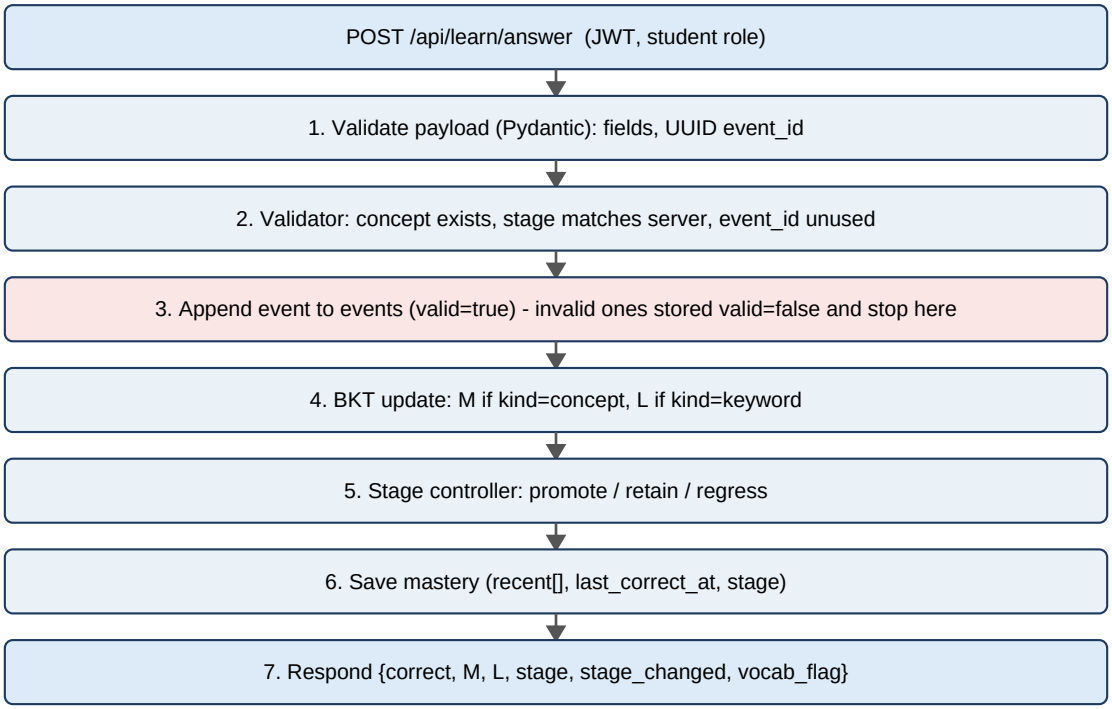
A pure function next_stage(M, L, recent, current_stage). It looks at concept mastery, keyword mastery and recent answers, not at English fluency.

Decision	Rule (placeholder thresholds, configurable)
Promote	M >= 0.85, and (current stage below 1, or L >= 0.70), and last 3 answers correct
Regress	M < 0.40, or last 3 answers incorrect
Retain	Anything else. Result is clamped to stages 0 to 4.
Vocabulary-support flag	M high but L low: do NOT regress. Keep the stage and flag 'needs vocabulary support'.

7.3 Event validator

An event is rejected, stored with valid=false and never used for mastery if any of these hold: a field is missing, the concept is unknown, the stage does not match the server record, or the event_id already exists.

7.4 Answer-submission pipeline



8. API design

All routes use JWT authentication and a role_required decorator. Errors use one consistent JSON shape. An OpenAPI file (backend/openapi.yaml) is written first so the frontend can build against mocks.

Method	Endpoint	Role	Purpose
POST	/api/auth/login	all	Returns a short-lived JWT
GET	/api/auth/me	all	Current user and role
GET	/api/concepts?subject=	student	Concept list with the student's stage and mastery
GET	/api/learn/{concept_id}	student	Content in the student's language at the current stage, plus keywords
POST	/api/learn/answer	student	Validate, log event, update BKT and stage; returns result
GET	/api/learn/revision	student	Concepts with low mastery or recent mistakes
GET	/api/teacher/class/{id}/summary	teacher (own class)	Weak concepts, stage distribution, vocabulary-support list
GET	/api/teacher/class/{id}/heatmap	teacher (own class)	Students by concepts matrix of mastery
GET	/api/teacher/student/{id}	teacher (own class)	One student's history and stage changes
POST	/api/teacher/attendance	teacher	Mark attendance
GET	/api/parent/child/{id}	parent (linked child)	Attendance and progress only
CRUD	/api/admin/users, /classes, /curriculum	admin	Manage users, classes, content and review status
GET/PUT	/api/admin/settings	admin	BKT parameters and thresholds
GET	/api/admin/events, /audit	admin	Event and audit log viewers
GET	/api/health	public	Database connectivity check
ANY	/api/tutor	student	Stub returning 501 Not Implemented in Phase 1

9. Dashboards and their features

Every dashboard shares a common base: login and logout, UI language switcher, role-based access, responsive layout for low-end phones, and clear loading and error states. Features marked P2 belong to Phase 2.

9.1 Student dashboard

Feature	Phase	Details
Summary	1	Name, grade, language, concepts completed and in progress
Concept list	1	Subject, chapter, lesson, concept with StageBadge and MasteryBar
Stage note	1	Visible line: the stage shows the amount of English support, not proficiency
Lesson view and keyword cards	1	Content at the current stage; English term beside the regional word from Stage 1
Quiz	1	Concept and keyword questions, instant feedback, message on stage change
Progress view	1	'Understanding' (M) and 'English words' (L) in plain labels
Vocabulary prompt and revision list	1	Shown when M is high and L low; concepts needing revision
Tutor chat	P2	Any language, sources shown, 'not enough evidence' reply
Offline and sync status	P2	Cached lessons, queued answers, tutor unavailable offline

9.2 Teacher dashboard

Feature	Phase	Details
Class overview	1	Students, average mastery, weekly activity
Student list	1	Stage and mastery per concept, with search and filter
Concept heatmap	1	Students by concepts, coloured by mastery
Stage distribution	1	Number of students at Stage 0 to 4 per concept
Weak concepts and vocabulary-support lists	1	Where many students struggle; students with high M and low L
Student detail	1	History, recent answers, stage changes
Attendance entry	1	Feeds the parent view
Assignments, alerts, recommendation reasons, tutor-flag review, CSV export	P2	Alert thresholds set to avoid alert overload

9.3 Parent dashboard

Feature	Phase	Details
Child selector	1	For parents with more than one linked child
Attendance	1	Calendar or percentage
Progress and stage overview	1	Plain language, with the English-support note
Recent activity	1	Last login and last lesson only; no answer-level detail
Notices and support tips	P2	School announcements; simple practice suggestions

9.4 Admin dashboard

Feature	Phase	Details
System overview	1	Counts of users, classes, concepts; health status
User and class management	1	Create, edit, deactivate, reset password, link parents to students
Curriculum content editor	1	Per language and per stage (s0 to s4), keywords, quiz questions
Content review status	1	Draft, reviewed, approved (covers TODO_REVIEW content)

Feature	Phase	Details
Settings	1	BKT parameters and thresholds, labelled untuned
Consent and privacy controls	1	Consent status, data export and delete requests
Audit log and event viewer	1	Who changed what; valid and invalid events
Performance, tutor monitoring, lesson pack manager	P2	Latency, errors, sync success, flagged answers, pack versions

Wording rule: use 'needs support', never 'weak student' or 'low ability'. Missed work can come from poor internet, shared devices or lack of time, as the paper itself notes.

10. Frontend architecture

Concern	Design
Routing	React Router with ProtectedRoute(role). Each role has its own route tree and layout.
API access	One Axios client with interceptors: attach JWT, handle 401 by redirecting to login, normalise errors.
State	React hooks and context for auth and language; server data fetched per page. No heavy state library needed.
i18n	i18next with ta.json, ml.json, kn.json, te.json, en.json. Lesson content comes from the API; UI text from the files.
Key components	StageBadge, MasteryBar, KeywordCard, QuizCard, ConceptHeatmap, StageDistributionChart, LanguageSwitcher.
Quiz behaviour	Generates a client UUID per answer (becomes event_id), shows feedback and any stage change returned by the API.
Accessibility	Labels, strong contrast, keyboard navigation, readable script fonts.
Low-end devices	Code splitting per role, small bundles, responsive layout, minimal animation.
Component tests	Vitest tests for StageBadge, KeywordCard and QuizCard.

11. Security, privacy and compliance

Area	Measure
Authentication	bcrypt password hashing, short-lived JWTs, login rate limiting
Authorisation	role_required on every route. Teachers reach only their own class; parents only linked children.
Parent data limit	Attendance and progress only. No interaction detail and no tutor conversations.
Input safety	Pydantic validation on every body; parameterised queries; consistent JSON errors
Secrets	Environment variables and GitHub Secrets; .env.example holds fake values; gitleaks in CI
Data minimisation	Collect only what the product needs; consent flag per student
Test data	Synthetic users only; no real student data in dev, test or staging
Compliance awareness	Digital Personal Data Protection Act 2023 (India), with institutional approval before any pilot
Transport and storage	HTTPS in staging and production; separate databases per environment
Auditability	audit_log for admin and content changes; event log retained for debugging

For minors, confirm with the institution how parental consent is to be collected before any real students use the system.

12. CI/CD and environments

Two developers work trunk-based on a protected main branch. Short-lived feature branches go through pull requests; CI must pass and the other developer must approve before merge.

Build once, promote the same image. Failed smoke test on /api/health stops the pipeline and keeps the previous version.

Rollback: redeploy the previous image tag (keep the last 3).

12.1 Team workflow

Item	Setup
Branching	Trunk-based: main plus short branches (feat/..., fix/...), 1 to 3 days each
Protection	No direct pushes; PR required; CI green; 1 approval from the other person
Ownership	CODEOWNERS: /backend to Bhavan, /frontend to Vishak; the other reviews
Commits	Conventional commits (feat:, fix:, test:)
Contract	openapi.yaml agreed first so frontend and backend work in parallel
Tracking	GitHub Projects board: Todo, Doing, Review, Done

12.2 CI jobs (on every pull request)

Job	Steps
backend-ci	Python 3.12, install, ruff, pytest with coverage against a MongoDB service container
frontend-ci	Node 20, npm ci, ESLint, Vitest, vite build
docker-build	Build both images; no push on pull requests

Job	Steps
security	pip-audit, npm audit, gitleaks

12.3 CD and environments

Environment	Hosting	Database	Deploy trigger
Local	Docker Compose	Local Mongo container	Developer runs docker compose up
Staging	Container host (e.g. Render, Railway, Fly.io); frontend on Vercel or Netlify	Atlas free cluster, separate DB	Automatic on merge to main, then smoke test on /api/health
Production	Same host type, separate service	Separate Atlas DB	Git tag v*, manual approval via GitHub Environments

A single VPS running Docker Compose behind Nginx is an equally valid option. The same images and pipeline apply, with SSH deploy (docker compose pull and up -d).

12.4 Configuration and secrets

- GitHub Secrets per environment: MONGO_URI, JWT_SECRET, deploy tokens (later the LLM API key).
- BKT parameters and stage thresholds in config.py, overridable by environment variables, so tuning needs no rebuild.
- Observability: /api/health, structured JSON logs, optional Sentry and an uptime check.

13. Testing strategy

Test output is exported as CI artifacts, so it can feed the functionality test table required in Section VIII-A of the paper.

Level	Tool	What is covered
Unit	pytest, Vitest	BKT bounds and updates; promote, retain, regress, clamp; vocabulary-support case; validator rules
API	pytest with Flask test client	Auth, RBAC (parent cannot call teacher routes), duplicate event_id changes nothing, full answer pipeline
Component	Vitest	StageBadge, KeywordCard, QuizCard
End to end	Playwright (small set)	Login, lesson, quiz, stage change, teacher dashboard update
Staging smoke	curl in CD	/api/health, login, fetch a concept

Quality gates

- Coverage of at least 85% on backend/services/.
- A simulated student must move from Stage 0 to Stage 2 over about 12 answers in a test.
- Functionality table columns: test ID, pre-condition, input, expected output, actual output, result, severity (critical, major, minor).
- Report tests run and percentage passed on the final build.

14. Phase 2 extension architecture

Phase 1 leaves clean hooks so these features are added without rewrites.

Feature	How it plugs in	Key design points
RAG tutor	Fill the /api/tutor blueprint. Add an ai/ module with embeddings and a vector index (Atlas Vector Search or FAISS).	Query uses language, subject, chapter, lesson, concept. Cosine similarity for retrieval. Prompt asks for an explanation at the student's stage with key English terms, and a statement when evidence is insufficient. Sources are always shown. Multilingual embedding model (for example MuRIL or a multilingual Sentence-BERT) chosen after testing on Roman-script and code-mixed questions.
Support score P(u,c)	New function over mastery plus last_correct_at and recurring-issue counts	Vector X = [M, L, recent assessment, time since success, recurring issues]. Output low, medium or high support with a reason shown to teachers.
Alerts and recommendations	Rule layer reading mastery and events	Thresholds chosen to avoid alert overload; every recommendation carries an explanation.
Offline lesson packs	Service worker plus IndexedDB; pack builder in admin	Cached lessons readable offline. The hosted tutor is NOT available offline, and the UI says so.
Sync queue	POST /api/events/sync (batch)	Client-generated event_id and the unique index make sync idempotent. Events are applied in order; duplicates are ignored.

Phase 2 data additions: embeddings collection (concept passages), tutor_logs (flagged answers), lesson_packs (versions).

15. Delivery roadmap

The plan assumes a 16-week semester. Shift the dates to match your calendar; your semester exams start on 11 November, so the build should begin after exams.

Phase	Weeks	Goal	Deliverable
1. Core MVP	1-6	Working learning loop	Login, concepts, lessons at current stage, quiz, BKT, stage controller, four dashboards (basic), Tamil content
2. Intelligence and offline	7-11	Tutor and low-connectivity support	RAG tutor, support score, alerts, lesson packs, sync queue
3. Evaluation and report	12-16	Evidence and documentation	Test tables, tutor review, performance tests, small approved pilot, ablation, final paper

Phase 1 build order

Week	Tasks
1	Requirements, repo, Docker Compose, Flask skeleton, Mongo connection, CI with a trivial test
2	Auth, JWT, RBAC; curriculum schema and seed script (1 subject, 8 concepts, Tamil)
3	Student flow: concept list, lesson at current stage, KeywordCard, StageBadge
4	BKT engine, stage controller, validator, with unit tests
5	Answer endpoint pipeline, event log, teacher dashboard
6	Parent and admin basics, other three languages (drafts), Review 1 demo

Work split

Person	Main ownership
Bhavan Kalyan S	Backend, BKT and stage controller, database, CD workflows, RAG tutor in Phase 2
Vishak P	Frontend, dashboards, i18n, frontend CI, offline sync in Phase 2
Both	Curriculum content, testing, documentation, reviews of each other's pull requests

Person	Main ownership
Dr. Sakthivel K	Guidance, approach review, pilot approvals

16. Risks, limitations and open decisions

Risks and mitigations

Risk	Mitigation
Writing content in four languages takes too long	Author Tamil fully first; machine translation plus teacher review for the rest, marked TODO_REVIEW
Tutor gives incorrect answers (Phase 2)	Retrieval grounding, insufficient-evidence reply, source display, teacher flags
Quality differs between languages	Evaluate and report each language separately
BKT defaults are wrong	State them as untuned; run a sensitivity analysis
No pilot approval in time	Use a clearly labelled simulated-learner evaluation instead
Offline sync bugs	Client event IDs, unique index, idempotent batch endpoint
Two-person team bottleneck	Monorepo, CODEOWNERS, OpenAPI contract, short branches

Limitations to state openly

- No controlled study exists, so no improvement in learning is claimed.
- Stage thresholds and BKT parameters are placeholders and must be tuned on representative data.
- Results may not generalise across schools, grades, subjects or languages.
- Offline mode supports cached lessons and local activity only; the hosted tutor needs a connection.
- Code-mixed and Roman-script questions may be handled less well by some models.

Open decisions for the team

- Which subject and grade for the first curriculum (the plan assumes Mathematics, Grade 9).
- Hosting choice: container platform plus static host, or a single VPS.
- How parental consent is collected for minors, agreed with the institution.
- Which embedding model is used for multilingual retrieval in Phase 2, decided after a small retrieval test.
- Paper clean-up before review: remove or relabel Fig. 3 and Fig. 4 (they show numbers although no data exists), change 'ablation was performed' to 'will be performed', and fix the citation numbers that do not match the reference list.